

Amortization Engine Implementation Walkthrough

How the Go port computes loan amortization: the field-presence dispatch, the faithful structural DOS port that is now the default engine, the piecewise fallback, the backward solvers, and the math — with the DOS lineage each piece preserves.

Scope. The Go amortization package `internal/finance/amortization/` (the structural DOS port, the piecewise engine, the backward solvers, first-pass, schedule-oracle bisection, advisories, types, validation), the API wiring in `internal/api/handlers.go`, and the shared math in `internal/finance/interest/` and `internal/dateutil/`.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference in `internal/` for verification.

Lineage. The Go code is a faithful port of the DOS Pascal engine; its comments cite the Pascal originals (e.g. [AMORTOP.pas:723](#)). For the original algorithm, see the companion `legacy/code_docs/amortization_dos_walkthrough.pdf`. This document explains the Go implementation.

Contents

0 How to read this document

1 Architecture: request to result

1.1 The package map · 1.2 Pointer fields → field presence · 1.3 The status ladder

2 The data model

3 First pass & dispatch

4 The two engines & the gate

4.1 `dosPortCanHandle` · 4.2 The structural DOS port · 4.3 `ComputeNext`, step by step

5 The piecewise fallback engine

5.1 Rate primitives · 5.2 The simple loop · 5.3 In-advance settlement · 5.4 The fancy engine · 5.5 Interest models

6 Backward solving

6.1 Closed-form estimates · 6.2 Schedule-oracle bisection · 6.3 The DOS secant port · 6.4 `inBackwardSolve` isolation

7 The advisory layer

8 The math primitives

9 Porting notes & gotchas

10 Appendix: call graph & file:line index

0 How to read this document

The Go port reproduces the Per%Sense amortization logic behind a REST API. Like every Per%Sense screen it is a **field-presence solver**: the caller fills in what is known, leaves one field blank, and the engine solves for the

blank — a payment, a rate, a loan amount, or a term. The forward direction generates a period-by-period repayment schedule; the backward direction inverts that schedule.

The big architectural change since the previous edition: the Go port now ships a **faithful structural port of the DOS engine** as the **default** for advanced-option ("fancy") loans. The original piecewise engine in `engine.go` remains as the entry point and required fallback for everything the port does not yet cover.

Citation convention

References look like `engine.go:1264` and resolve under `internal/finance/amortization/` unless another path is named (e.g. `interest/math.go:134`, `api/handlers.go:752`). Where the Go code cites a Pascal line, that lineage is noted as `(AMORTOP.pas:723)`.

The three functions to anchor on. `engine.go:139 Amortize(input)` is the package entry — it classifies, then *either* delegates to the structural port or runs the piecewise path. `dosport_entry.go:424 AmortizeDOS(input)` is the faithful port — one schedule walk, one Newton solve. `engine.go:1264 generateFancySchedule` is the piecewise period-by-period heart used as both fallback and backward-solve oracle.

1 Architecture: request to result

1.1 The package map

```
POST /api/amortization/calc
  | HandleAmortizationCalc                                api/handlers.go:752
  v decode AmortizationRequest (pointer fields)
build LoanInput (+ any advanced option => Fancy=true)    api/handlers.go:329 / :1081
  |
  v amortization.Amortize(input)                          engine.go:139
FirstPass ---> classify term/dates                       firstpass.go:43
ComputeTrueRate / GrowthPerPeriod                       engine.go:44 / :29
if dosPortCanHandle(...) => AmortizeDOS(input)           engine.go:226 / dosport_entry.go:424
else: solve unknown + generate (piecewise)              backward.go / engine.go
  |
  v AmortResult{Schedule, totals, APR, Advisories} types.go:175
```

The files: `engine.go` (entry, the gate call, piecewise schedule generators, rate primitives), `dosport.go` / `dosport_entry.go` / `dosport_walk.go` (the structural DOS port), `backward.go` (closed-form solvers), `fancybisect.go` (schedule-oracle bisection + the DOS secant port), `firstpass.go` (term/date classification), `types.go` (model), `validate.go`, `advisories.go` (warnings).

1.2 Pointer fields → field presence

The API request uses pointer types for every field that participates in field-presence dispatch, so a missing JSON key is distinguishable from a supplied zero `api/handlers.go:180,278`:

```
Amount *float64 `json:"amount,omitempty"` // nil => blank => solve the loan amount
Rate   *float64 `json:"rate,omitempty"`   // nil => blank => solve the rate
```

The handler only stamps `InOutInput` when the pointer is non-nil `api/handlers.go:858,862`; a nil pointer leaves the status at Go's zero value `StatusEmpty` (`=0`). A blank `Payment` arrives as a JSON zero rather than a

pointer, so it is explicitly downgraded to `StatusEmpty` `api/handlers.go:903`. Supplying any advanced option flips `input.Fancy = true` `api/handlers.go:329`.

1.3 The status ladder

Each field carries an `int8` status forming the dispatch ladder `types/constants.go:54–58`: `InOutBad(-1) < StatusEmpty(0) < InOutOutput(1) < InOutDefault(2) < InOutInput(3)`. The pervasive guard `Status >= InOutDefault` means "known" (a default or a user input); `Status < InOutDefault` means "blank — solve for it." A solved field is written back as `InOutOutput` (or promoted to `InOutInput` before a forward run).

2 The data model

The input bundle is `LoanInput` `types.go:145`: a `Loan`, the advanced-option slices, `Settings`, and the `Fancy` flag. The core loan `types.go:43` pairs each value with a status:

Field	Meaning
Amount	loan principal
LoanDate / FirstDate	closing date / first regular payment date
LoanRate	nominal rate as a yield at <code>PerYr</code> (the engine converts to a continuous rate)
NPeriods / LastDate	term as a count or an end date (mutually exclusive; <code>LastOK</code> tracks which is known)
PerYr / PayAmt / Points	payments per year / regular payment / APR points

Advanced options `types.go:70–125`: `BalloonPayment` (date + amount; a date-only balloon is a target to solve), `RateAdjustment` (ARM: date + optional new rate and/or payment), `Prepayment` (extra periodic stream with a start, a count or stop-date bound, a frequency, and a payment), `Moratorium` (interest-only until `FirstRepay`), `Target` (minimum principal reduction per payment), and `SkipMonths` (a string like "6-8,12" parsed into a 13-element month set). The output is `AmortResult` `types.go:175`: the `Schedule` (one `PaymentRecord` per row — date, payment, interest, remaining principal, cumulative interest), totals, optional APR, the echoed post-`FirstPass` term/dates, resolved balloons, and the `Advisories` list (§7).

3 First pass & dispatch

`FirstPass` `firstpass.go:43` derives whichever of {first date, last date, #periods} is blank but computable, in three field-presence arms (no iteration): default the first payment date from the loan date + frequency (`Amortize.pas:244`); derive `lastDate = AddNPeriods(firstDate, perYr, n-1)` when first + count are known `firstpass.go:85`; or derive `n = NumberOfInstallments(first, last)` when first + last are known `firstpass.go:102–105`. The count is computed on a **forced 360 basis** `firstpass.go:104–105` to match the DOS `NumberOfInstallments`. `LastOK` records whether the term is now known — the central flag the dispatch reads.

`Amortize` `engine.go:139` then validates required fields, computes the rate and growth factor, and reaches the central fork:

```
// engine.go:226 – the gate that picks the engine
if dosPortCanHandle(input, loan, &settings) {
    pin := input
    pin.Loan = loan // post-FirstPass (term/dates derived)
    return AmortizedDOS(pin)
}
// ...else fall through to the piecewise engine below
```

When the gate declines, the piecewise path forces fancy mode where its simple loop cannot cope (exact-daily, or USA-rule on exact/non-360 — `engine.go:235,240`), determines the blank field, solves it, and branches to a generator:

Unknown (blank field)	Piecewise solver	Line
Term (payment known, term blank)	<code>solveFancyTermFromPayment / solveNPeriodsFromPayment</code>	<code>backward.go:945 / :380</code>
Payment (blank PayAmt)	<code>estimatePayment → refine (solveFancyPayment / dosIteratePayment)</code>	<code>engine.go:852; fancybisect.go:316,114</code>
Amount / Rate	solved in the API handler before <code>Amortize</code> runs	<code>api/handlers.go:1124 / :1135</code>

The amount/rate solves run in the handler on a throwaway `FirstPass`'d copy (so the solver guards see derived dates), then write the solved value back as `InOutInput` so the subsequent `Amortize` runs forward normally `api/handlers.go:1124–1147`.

4 The two engines & the gate

For an advanced-option loan, the piecewise engine historically solved each segment with a separate **sub-loan**, which is not identical to the real schedule's tail; stacked options (multi-ARM × balloon × skip × moratorium) could therefore drift by cents. The structural DOS port instead runs **one** schedule walk and solves every unknown with **one** Newton over it, exactly as DOS does — so the combinations compose correctly by construction `dosport.go:12–17`.

What changed. The structural port (`dosport*.go`, var `dosPortEnabled = true` at `dosport_entry.go:246`) is now the **default** for the validated advanced-option domain, where it matches the real DOS engine to the cent. The original piecewise `Amortize` is **not** deleted — it remains the entry point and the required fallback for everything the gate excludes. The port is validated to **zero divergence vs the real DOS engine across ~38,000 fuzzed cases** (the `dosport_fuzz_*` sweeps over frequency, first-period, prepaid, basis, given-payment, and the merged option cube).

4.1 dosPortCanHandle — the gate

`dosPortCanHandle` `dosport_entry.go:269` returns `true` only inside the validated domain. It declines (falling back to piecewise) for any of:

Fallback condition	Line
Port disabled, not fancy, or <code>inBackwardSolve</code> (a piecewise backward solver is using <code>Amortize</code> as its oracle)	:270
<code>NPeriods > MaxSchedulePeriods</code> (the 10000-row safety bound)	:274
In-advance, Rule-of-78, exact-daily, or daily mode	:278
Amount or rate is the unknown (the port solves/uses only the payment)	:281
REPLACE-mode extras (! <code>PlusRegular</code> with a balloon/prepayment that <i>replaces</i> rather than adds)	:292
An unbounded / unsupported prepayment shape; AO10 duration is allowed but routed carefully	:303
AO2 target-balloon with a blank payment (under-determined); an off-cycle balloon (date not on a payment date)	:325, 338
AO6/AO7 adjustment combined with a balloon dated after it (the one deliberate DOS-bug divergence)	:381
Every calendar month skipped (the loan never amortizes)	:405

The off-cycle-balloon and all-month-skip exclusions keep the piecewise engine's Rev-10 exact-date balloon draining and its explicit "does not retire" handling in force, where the port would otherwise apply an off-cycle balloon at the next payment.

4.2 The structural DOS port

`AmortizeDOS dosport_entry.go:424` mirrors DOS `MakeTable`: solve the blank payment when one is unknown (`estimateAndRefinePayment dosport_entry.go:177`), then build the schedule with `repayFancyLoan(entire)`. The engine state lives in a `dosEng dosport.go:82` that holds the Pascal module-level "globals" (`h^`, `balloon[]`, `pre[]`, `adj[]`, `mor`, `targ`, `skp`, the running counters) as one instance per call — pure and testable rather than real package globals. One row in progress is a `dpPayment dosport.go:67`, a faithful copy of the Pascal `paymenttype` object (`AMORTOP.pas:87`).

The port reproduces the DOS procedures one-for-one:

Go (dosport*)	Pascal original	Role
<code>computeNext dosport.go:213</code>	<code>AMORTOP.pas:723</code>	advance one period (§4.3)
<code>findNextExtra / checkOffBalloon dosport.go:132,178</code>	<code>AMORTOP.pas:629,694</code>	locate & consume the next balloon/prepay
<code>repayFancyLoan dosport_walk.go:72</code>	<code>AMORTOP.pas:1331</code>	the full period-by-period walk
<code>iterate dosport_walk.go:179</code>	<code>AMORTOP.pas:1690</code>	generic Newton over a <code>var x</code> (drive terminal balance to 0)
<code>reAmortize dosport_walk.go:250</code>	<code>AMORTOP.pas:1783</code>	ARM rate/payment change mid-walk
<code>solveUnknownBalloon / solveUnknownPrepay dosport_walk.go:344,364</code>	<code>Amortize.pas:784,827</code>	backward solves over the same walk
<code>saveState / restoreState dosport_walk.go:38,49</code>	<code>AMORTOP.pas:208,239</code>	snapshot/roll-back cursors so a speculative <code>iterate</code> walk can be undone

4.3 ComputeNext, step by step

`computeNext(pt, p, usap)` [dosport.go:213](#) advances one period in the exact order DOS does ([AMORTOP.pas:723–822](#)). The piecewise `generateFancySchedule` (§5.4) mirrors the same six steps:

1. **Advance the date** — step the regular grid one period from `base_date`, anchored to the first-payment day-of-month.
2. **Skip-month zeroing** — if the month is in the skip set, set `payamt := 0`, else the base payment `d`.
3. **Find the next extra** — `findNextExtra` returns the soonest balloon and/or co-dated prepayment (summed); `balloonpos` positions it before (−1: pay only the extra, do *not* advance the regular grid), coincident (0: add or replace `per_plus_regular`), or after (1). The regular grid advances only when a regular payment occurred (`balloonpos >= 0`).
4. **Interest interval & accrual** — on the 360 basis or non-exact mode the interval snaps to whole calendar months; otherwise it is the exact day-count. Interest accrues on the principal *net of the USA bucket* (`p - usap`), rounded to the cent only when the payment is hard.
5. **Moratorium & target** — during a moratorium only interest is collected; otherwise, if the period's principal reduction falls short of `targ.target`, the payment is bumped up to meet it. **Target overrides skip-month zeroing** ([AMORTOP.pas:653](#)).
6. **New balance & USA rule** — `p := p + interest - payamt`; the USA bucket tracks unpaid interest `usap := usap + interest - payamt`, floored at 0, so principal whose interest is uncovered does not itself earn interest.

5 The piecewise fallback engine

When the gate declines, `Amortize` runs the original piecewise engine. It is the required fallback (and the oracle the backward solvers drive — §6).

5.1 Rate primitives

All generators share: `GrowthPerPeriod` [engine.go:29](#) returns the per-period factor $f = 1 + \text{rate}/\text{RealPerYr}(\dots)$ (weekly/biweekly use an explicit 7- or 14-day fraction); `ComputeTrueRate` [engine.go:44](#) derives the continuous rate for daily accrual; `firstPeriodProrate` [engine.go:812](#) measures the odd first period as a fraction of a normal period; `periodYearFraction` [engine.go:838](#) is the interior-period analogue; `exactDaily` [engine.go:20](#) gates true daily accrual on `Exact && Basis != 360` ([AMORTOP.pas:765](#)).

5.2 The simple loop

`generateSimpleSchedule` [engine.go:875](#) handles plain (non-fancy) loans. A user-supplied (hard) payment triggers the "Dav Holle provision": per-period interest is rounded to cents [engine.go:888–891](#) ([Amortize.pas:1687](#)). The first-period window emits a settlement stub for prepaid loans or prorates the first period's interest. The per-period order of operations:

```

interest = p*(f-1)           (interior, arrears)      engine.go:1030
interest = p*(f-1)*prorate   (odd first)
daily:   interest = p*(e^(truerate*YearsDif) - 1)
in-advance: interest = (p-pmt)*(f-1)/(2-f)          engine.go:1031
new balance: p = p + interest - pmt                 engine.go:1031

```

Rule-of-78 (360 basis, non-in-advance) front-loads interest via a constant step $r78step = (n \cdot \text{payment} - \text{amount}) / (0.5 \cdot n \cdot (n+1))$ decremented each period `engine.go:1011–1022` (`Amortize.pas:1414`). On the final period or when the balance would go non-positive, the residual is folded into the payment `engine.go:1100–1109`.

5.3 In-advance settlement

`generateExactInAdvanceSchedule` `engine.go:1183` handles DOS's distinct exact-in-advance shape: a row-0 settlement-interest row dated the loan date, a one-period base shift, then $n-1$ actual-day amortizing rows (`AMORTOP.pas:1148–1240`). The simple in-advance branch in `generateSimpleSchedule` `engine.go:958–975` emits the equivalent settlement row so the in-advance total is not short by the settlement interest ($\approx \$500$ on a 100k/6% monthly loan, the former bug).

5.4 The fancy engine

`generateFancySchedule` `engine.go:1264` is the piecewise period-by-period engine for any advanced-option loan the gate routed here, with a USA-rule exempt-principal bucket `usap.veryLast` `engine.go:1291` sets the schedule terminus (latest of last date, balloons, prepay stops). The main loop mirrors DOS `ComputeNext` (`AMORTOP.pas:723–822`):

1. **Off-cycle draining** `engine.go:1427–1540`: any balloon/prepayment dated before the regular date is emitted as its own dated row with partial-period interest from `prevDate` — applied at its exact date, not folded into the next payment.
2. **Period interest** `engine.go:1564–1576` on $(p - \text{usap})$, rounded if hard `engine.go:1579`.
3. **Skip-month zeroing** `engine.go:1585`, then moratorium (interest-only) `engine.go:1603`.
4. **Coincident extras** (balloons + prepayments on this date; `PlusRegular` adds vs replaces) `engine.go:1643–1705`.
5. **Target**: bump payment to meet the minimum principal reduction — overriding skip-month zeroing `engine.go:1725` (`AMORTOP.pas:653`).
6. **New balance + USA bucket** `engine.go:1800–1802`, then rate adjustments at ARM dates (re-amortize remaining balance, or solve the implied rate from a new payment) `engine.go:1846–1908`.

5.5 Interest models in one place

Model	Per-period interest	Where
Arrears (simple/compound)	$p \cdot (f-1)$, or actual-day $p \cdot \text{rate} \cdot \text{YearsDif}$	<code>engine.go:1030,1576</code>
Daily (continuous)	$(p - \text{usap}) \cdot (e^{\text{truerate} \cdot t} - 1)$	<code>engine.go:1574</code>
In-advance (annuity-due)	$(p - \text{pmt}) \cdot (f-1) / (2-f)$	<code>engine.go:1031</code>
Rule of 78	decrement by <code>r78step</code> each period	<code>engine.go:1022</code>
USA rule	accrue on $(p - \text{usap})$; bucket floored at 0	<code>engine.go:1574,1802</code>

6 Backward solving

6.1 Closed-form estimates

Each piecewise solver starts from annuity algebra. `SolvePaymentClosedForm` `backward.go:83`: $\text{pay} = \text{amount} \cdot (f-1)/(1-f^{-n})$ via `Lnn/Exxp`, adjusted for first-period proration and in-advance. `SolveLoanAmount` `backward.go:197`: $\text{amount} = (1-f^{-n})/(f-1) \cdot d + \sum \text{balloon} \cdot e^{-\text{rate} \cdot \text{yrs}}$. `SolveRate` `backward.go:298`: Newton/secant on the repay residual, first guess $\text{payAmt} \cdot \text{perYr} / \text{amount}$ floored at 0.02. `solveNPeriodsFromPayment` `backward.go:380`: $n = \text{round}(1.4999 + \ln(1-p_1(1-ff)/(ff \cdot d))/\ln(ff))$ (`AMORTOP.pas:1660`). Balloon and prepayment solves live at `backward.go:519,651`; the prepayment-duration solve at `backward.go:827`.

6.2 The schedule-oracle bisection

For fancy loans the closed form is only an estimate; refinement runs against the real forward schedule. Because the engine forces the final payment and stops early, the unforced residual is discontinuous and Newton on it is unstable — so `fancybisection.go` uses bisection `fancybisection.go:10–28`. `fancyOverUnder` `fancybisection.go:166` runs `Amortize` and classifies a trial as under-amortized (+1), over-amortized (−1), or solved (0). `fancyBisect` `fancybisection.go:217` expands the bracket until the sign straddles, then bisection to a tolerance (5e-4 for amount/payment, 1e-7 for rate). `solveFancyAmount / Rate / Payment` `fancybisection.go:275,293,316` wrap it with the unknown pinned (rate capped at 200%).

6.3 The DOS secant port

`dosIteratePayment` `fancybisection.go:114` is a bug-for-bug port of DOS `Iterate` (`AMORTOP.pas:1690`), driving `repayExactTerminal` `fancybisection.go:43` (the unforced terminal balance) to zero by finite-difference secant $\text{newdelta} = \text{delta} \cdot p / (\text{final} - p)$, with the DOS divergence brake (`count += 5` when the step stops shrinking) and the faithful `bestx-after-update` ordering. Convergence: $|\text{residual}| < \text{halfpenny} = 0.005$ or 20 iterations.

6.4 inBackwardSolve isolation

The piecewise backward solvers (`SolveLoanAmount`, `SolveRate`, `SolveBalloonAmount`, `SolvePrepaymentAmount`) use `Amortize` itself as a forward oracle with trial values. To keep those recursive forward runs on the *same* piecewise engine (and avoid re-entering the port mid-search), each defers `beginBackwardSolve()` `backward.go:198,299,520,652`, which sets the package flag `inBackwardSolve` `dosport_entry.go:259`. The gate's very first check `dosport_entry.go:270` reads that flag, so any nested `Amortize` during a backward solve stays piecewise.

The structural heart of the backward path. The closed-form estimate gets the solver close; then the engine is used as its own forward oracle — `fancyOverUnder` calls `Amortize` recursively — and bisection/secant drives the real schedule's terminal balance to zero, the same criterion as DOS's `Iterate`.

7 The advisory layer

Both engines run their result through the same advisory pass so the UI can surface non-fatal warnings. `appendResultAdvisories` `advisories.go:54` inspects the finished `AmortResult` and the input and appends tiered messages via `AmortResult.add` `advisories.go:145` — e.g. a solved prepayment that retired the loan

early (A-W7), a balloon dropped because the payment was computed not input (A-W11), or an ARM whose implied rate is negative (A-W12, emitted once). `appendScheduleWarnings` `advisories.go:22` adds high-rate and rate-status notes. The structural port emits the same advisory codes so the two engines are indistinguishable to the API consumer (`dosport_advisory_diff_test.go` guards this).

8 The math primitives

The shared interest and dateutil packages reproduce the DOS primitives exactly — the financial behaviour is defined by these, not by the standard library.

Primitive	Behaviour	Location
<code>Round2(x)</code>	round-half-down: <code>halfpenny = 0.005</code> – teeny, add toward zero then truncate at 2 decimals. The –teeny sends an exact half-cent downward.	<code>interest/math.go:134</code>
<code>Exxp(x)</code>	safe e^x : overflow error at $x > 70$, $1e-32$ below -70 , cubic Taylor for $ x < 1e-4$.	<code>interest/math.go:43</code>
<code>Lnn(x) / Power(x,n)</code>	safe $\ln$ (Taylor near 1, for the Turbo precision bug) / $e^{n \cdot \ln x}$.	<code>interest/math.go:67 / :100</code>
<code>RealPerYr / YieldFromRate / RateFromYield</code>	periods-per-year; continuous $\leftrightarrow$ periodic-yield conversions $n(e^{r/n}-1)$ and $n \cdot \ln(1+y/n)$.	<code>interest/rates.go:42 / :63 / :79</code>
<code>YearsDif</code>	day-count years: 30/360 with end-of-month corrections, or actual 365/366 for loan calcs.	<code>dateutil/dateutil.go:643</code>
<code>AddPeriod / AddNPeriods / NumberOfInstallments / Julian</code>	date stepping by frequency; period counting; serial-date math (epoch 1900).	<code>dateutil/dateutil.go:442 / :526 / :720 / :126</code>

9 Porting notes & gotchas

- **The port is the default; piecewise is the fallback.** Do not assume a fancy loan ran the `engine.go` loop — check the gate `engine.go:226`. Outside the validated domain (in-advance, R78, exact, off-cycle balloons, AO6/AO7-with-balloon, all-month-skip, amount/rate unknowns, backward-solve oracles) it falls through to piecewise.
- **Exact requires a non-360 basis.** `exactDaily` is true only when both the `Exact` flag and a non-360 basis are set `engine.go:20`; on 360, `Exact` is a no-op (`AMORTOP.pas:765`).
- **Exact and USA-rule force fancy mode** `engine.go:235,240` — DOS routes every non-360 loan and every USA-rule loan through the period-by-period engine.
- **Odd-first payment is augmented, not just prorated.** The closed-form payment shortcut applies only to plain 360/prepaid/in-arrears/natural-first loans; otherwise the engine iterates. The schedule accrues whole-month interest while the payment is augmented — a deliberate DOS asymmetry, documented inline `engine.go:298–340`.
- **In-advance blank-payment loans iterate** via the annuity-due secant rather than the ordinary-annuity closed form `engine.go:563–598`.

- **Off-cycle balloons land at their exact date** with partial interest, not folded into the next payment
engine.go:1461–1540 (AMORTOP.pas:694).
- **Negative implied ARM rate is allowed** (overpaying payment), bounded $|rate| < 2$ — clamping at zero stalled the secant.
- **The fancyBisect-over-Newton choice is intentional** — the forced final payment makes the residual a step function; see the rationale block fancybisect.go:10–28.

10 Appendix: call graph & file:line index

```

POST /api/amortization/calc -> HandleAmortizationCalc      api/handlers.go:752
|- decode (pointer fields -> status)                      api/handlers.go:858
|- [Amount/Rate nil] backward solve on FirstPass'd copy
|   SolveLoanAmount -> solveFancyAmount                  backward.go:197 / fancybisect.go:275
|   SolveRate       -> solveFancyRate                    backward.go:298 / fancybisect.go:293
+- Amortize(input)                                         engine.go:139
|   |- FirstPass (defFirst / last / n)                    firstpass.go:43
|   |- ComputeTrueRate / GrowthPerPeriod                 engine.go:44 / :29
|   |- GATE: dosPortCanHandle?                            engine.go:226 / dosport_entry.go:269
|   |   yes -> AmortizeDOS                                dosport_entry.go:424
|   |   |   |- estimateAndRefinePayment                  dosport_entry.go:177
|   |   |   +- repayFancyLoan -> computeNext             dosport_walk.go:72 / dosport.go:213
|   |   |       +- (ARM) reAmortize / iterate            dosport_walk.go:250 / :179
|   |   no  -> piecewise:
|   |       |- [term] solveNPeriodsFromPayment | solveFancyTermFromPayment backward.go:380 / :945
|   |       |- [pmt] estimatePayment -> solveFancyPayment | dosIteratePayment engine.go:852 / fancy
|   |       +- generateSimpleSchedule | generateFancySchedule | exact-in-advance engine.go:875 / 12
|   |- ComputeAPRWithPoints                              backward.go:458
|   +- appendResultAdvisories                            advisories.go:54

oracle: every solveFancy*/dosIteratePayment re-runs Amortize via fancyOverUnder / repayExactTerminal
fancybisect.go:166 / 43

isolation: backward solvers set inBackwardSolve so nested Amortize stays piecewise
dosport_entry.go:259 / backward.go:198

```

Symbol	Reference
Amortize (entry / gate)	engine.go:139,226
AmortizeDOS / dosPortCanHandle / dosPortEnabled	dosport_entry.go:424 / 269 / 246
computeNext / repayFancyLoan / iterate / reAmortize	dosport.go:213 / dosport_walk.go:72 / 179 / 250
FirstPass	firstpass.go:43
generateSimpleSchedule / generateFancySchedule / exact-in-advance	engine.go:875 / 1264 / 1183
SolvePaymentClosedForm / SolveLoanAmount / SolveRate	backward.go:83 / 197 / 298
fancyBisect / dosIteratePayment	fancybisect.go:217 / 114
appendResultAdvisories	advisories.go:54
Round2 / Exxp / Lnn	interest/math.go:134 / 43 / 67
YearsDif / AddPeriod	dateutil/dateutil.go:643 / 442
API handler	api/handlers.go:752

Per%Sense Go port. Citations resolve under `internal/`. For the original DOS algorithm this engine ports, see `legacy/code_docs/amortization_dos_walkthrough.pdf`. Companion Go walk-throughs: mortgage, present value, actuarial.